

ShopEZ

Apparel E-Commerce Web Application Project Documentation

Technology

Academic

Year

MERN-Stack

2026–

2027

Prepared By
Arya Vijay Sakore

Introduction

Welcome to ShopEZ — an everyday essentials clothing storefront built to make online shopping for apparel simple, fast, and reliable. From browsing a curated catalog to managing size and color variants, checking out, and tracking an order, ShopEZ covers the complete shopping journey end to end.

Shoppers can browse products across multiple categories — T-Shirts, Shoes, Dresses, Jackets, and Accessories plus more in store coming— filter by category, and search by name. Each product supports multiple size and color variants, each tracked with its own independent stock count, so availability is always accurate down to the specific combination a customer wants.

With a clean, editorial-inspired interface, users can add items to their cart, adjust quantities, check out with a shipping address and payment method of their choice, and track every order's status from “pending” through to “delivered.” Customers can also leave star ratings and written reviews on products they've purchased, helping future shoppers make informed decisions.

Behind the scenes, a dedicated admin panel gives store administrators full control: adding and editing products with their variants, managing every order's fulfillment status, and configuring site-wide settings like the homepage banner image and the list of shopping categories — all without touching a line of code.

Scenario Based Case Study

Background

“ShopEZ” is a full-stack e-commerce web application designed to make buying everyday clothing simple and dependable. The platform offers a growing catalog across multiple categories, with size- and color-specific inventory tracking, and combines a straightforward shopping experience with a capable admin back-office for managing the store.

Scenario

User Persona:

- Name: Sarah Lopez
- Age: 24
- Profession: Graphic Designer, works long hours from home
- Goal: Refresh her everyday wardrobe with well-fitting basics without spending hours comparing sites
- Challenges: Limited free time to shop, frustration with sites where sizes/colors are unclear about stock, and cluttered checkout flows

The Challenge

Sarah had tried a few other online stores before but found them either confusing to navigate or frustrate at checkout — discovering an item was out of stock in her size only after adding it to the cart. She needed a store that could:

- Show real, accurate stock for the exact size and color she wanted
- Offer a fast, uncomplicated checkout with clear order tracking afterward
- Let her see what other buyers thought before purchasing

- Work reliably across categories, from T-shirts to jackets

Solution: How ShopEZ Helped

When Sarah visited ShopEZ, she was able to filter straight to the “T-Shirts” category and open a product to see live stock per size and color combination — no guessing whether “Medium” was available. The site gave her:

- **Variant-level accuracy:** Each size/color combination showed its own real-time stock count before she added it to her cart.
- **A fast checkout:** A single address and payment-method form, with a clear order summary and free-shipping threshold shown upfront.
- **Order tracking:** A profile page showing her order's status update from “pending” to “shipped” to “delivered,” without needing email support.

Within minutes, Sarah placed her first order and left a review on the product after it arrived — helping the next shopper decide with confidence.

System Requirements

To ensure smooth development, deployment, and usage of the ShopEZ web application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications.

1. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux — supports cross-platform development and testing.
- **Node.js (v18 or above):** Provides the runtime environment for the frontend to build tooling and power the Express backend.
- **npm (v9 or above):** Package manager used to install all frontend and backend dependencies.
- **React.js + Vite:** React is the UI library; Vite is the build tool and dev server used to scaffold and run the client.
- **Tailwind CSS:** Utility-first CSS framework used for all styling.
- **React Router DOM:** Handles client-side routing across all pages.
- **Browser:** Google Chrome / Firefox (latest version) — for rendering and testing the UI in real time.
- **Express.js:** Lightweight Node.js web framework used to build the REST API.
- **Mongoose:** ODM (Object Data Modeling) library used to define schemas and interact with MongoDB.
- **MongoDB Atlas:** Cloud-hosted NoSQL database used to store all application data (users, products, carts, orders, reviews, site configuration).
- **JWT (jsonwebtoken) & bcryptjs:** Used for authentication — signing session tokens and hashing passwords.
- **Thunder Client / Postman:** Tools used for testing API endpoints during backend development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

2. Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — for running development servers, the code editor, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — required for package installations and local project files.
- **Display:** 1366x768 or higher — recommended for a comfortable coding and layout-review experience.

Technical Architecture

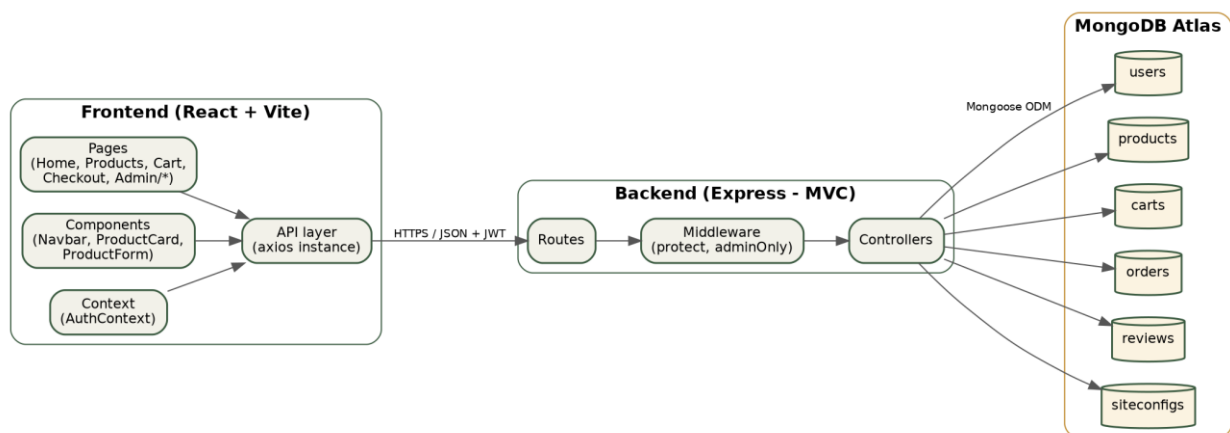


Figure 1: ShopEZ system architecture

Technical Architecture Overview

ShopEZ is built on a three-layer architecture that cleanly separates presentation, business logic, and data storage — a standard MERN-stack design that keeps the codebase modular and easy to extend.

Frontend (React + Vite): A single-page application composed of Pages (Home, Products, Cart, Checkout, Profile, and the full Admin section), reusable Components (Navbar, ProductCard, ProductForm, ProtectedRoute), and a global AuthContext that tracks the logged-in user across the app. All network calls flow through a single shared axios instance.

API layer / Axios instance: A configured axios client that automatically attaches the user's JWT to every outgoing request, so individual pages never have to handle authentication headers manually.

Backend — Routes (View layer): Express route files define every REST endpoint, grouped by resource: /api/users, /api/products, /api/cart, /api/orders, and /api/config.

Middleware: Two middleware functions gate access — protect verifies the JWT and attaches the logged-in user to the request, and adminOnly restricts a route to users with role: "admin".

Controllers: Contain all business logic — validating input, checking stock, calculating totals, and coordinating multiple models (for example, placing an order reads the Cart, validates Product stock, decrements it, and creates an Order in a single controller function).

Database — MongoDB Atlas: A cloud-hosted MongoDB cluster storing six collections: users, products, carts, orders, reviews, and siteconfigs, accessed through Mongoose schemas that enforce structure and validation.

ER Diagram

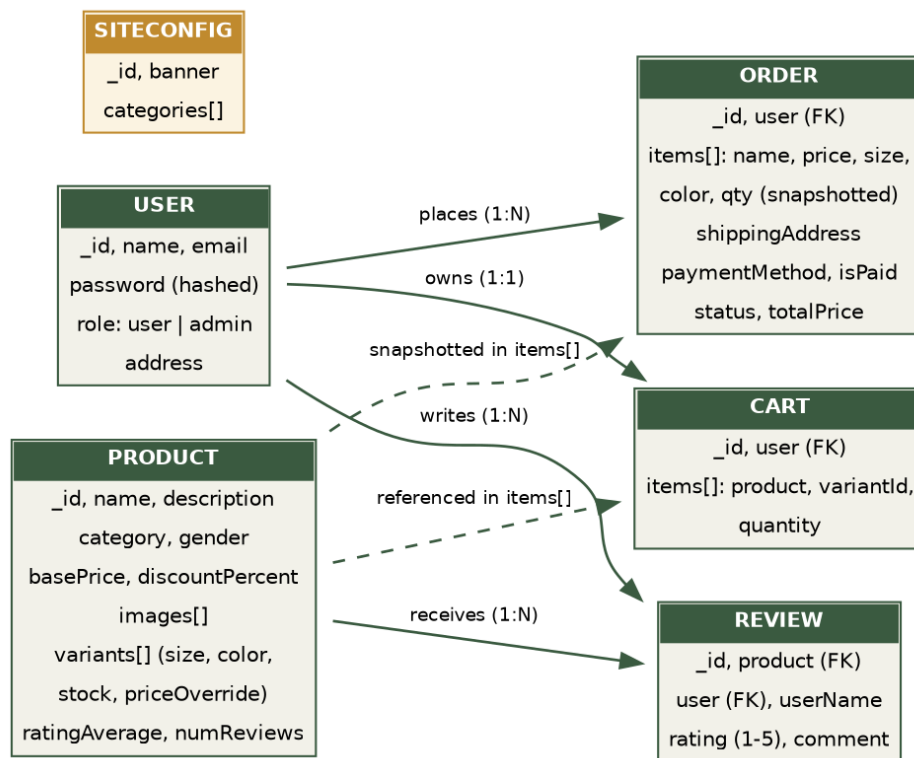


Figure 2: ShopEZ entity-relationship diagram

Entities and Relationships

1. User

- Represents both customers and administrators of the platform.
- Each user has a unique profile with name, email, a securely hashed password, an optional address, and a role (“user” or “admin”) that determines access to admin features.

Relationships:

- Owns → Cart: Each user has exactly one cart.
- Places → Orders: A user can place many orders over time.
- Writes → Reviews: a user can leave one review per product.

2. Product

- Represents an item in the catalog, with a name, description, category, gender, base price, optional discount percentage, and one or more images.
- Holds an embedded variants array — each variant defines a size/color combination with its own independent stock count and an optional price override.

Relationships:

- Referenced in → Cart items and Order items (by ID, with order items also storing a price/name snapshot).
- Receives → Reviews, which roll up into `ratingAverage` and `numReviews` on the product itself.

3. Cart

- One cart per user, holding an array of items — each referencing a specific product and variant, plus a quantity.
- Cleared automatically once its contents are converted into an order at checkout.

4. Order

- Created at checkout from the contents of a user's cart. Each item within an order snapshots the product's name, image, price, size, and color at the time of purchase — so later edits to a product never alter historical order records.
- Stores the shipping address, payment method, item/shipping/total price breakdown, and a status field that progresses from “pending” through “processing,” “shipped,” and “delivered” (or “cancelled”).

5. Review

- One review per user per product, storing a 1–5 star rating and a written comment.
- Automatically recalculates the parent product's average rating and review count whenever a review is added or removed.

6. SiteConfig

- A single, site-wide configuration document storing the homepage banner image URL and the list of managed categories — editable only by admins, and consumed by the storefront's category filters and product form.
-

Key Features

1. User Management

- **Registration & Authentication:** Users can create an account and log in securely; passwords are hashed with bcrypt and sessions are managed with signed JWTs.
- **Role-Based Access:** A single role field distinguishes regular users from admins, gating access to the entire admin panel.

2. Product Catalog Management

- **Category & Gender Filtering:** Products can be filtered by category and browsed by search term.
- **Size & Color Variants:** Every product supports multiple variants, each with independently tracked stock and an optional price override.
- **Discount Pricing:** Products can carry a discount percentage, applied consistently across the product card, detail page, cart, and final order total.

3. Cart & Checkout

- **Persistent Cart:** Each user's cart persists in the database, with add/update-quantity/remove functionality.
- **Stock-Safe Checkout:** Stock is re-validated at checkout time (not just at add-to-cart time) to prevent overselling, and is decremented atomically when an order is placed.
- **Flexible Payment Selection:** Users choose between Cash on Delivery, Card, or UPI at checkout.

4. Order Management & Tracking

- **Order History:** Users can view every past order and its current status from their profile.
- **Status Tracking:** Orders move through a defined lifecycle — pending → processing → shipped → delivered (or cancelled) — updated by admins.

5. Reviews & Ratings

- **Star Ratings & Comments:** Users can leave a 1–5 star rating with a written review on any product.
- **Automatic Rating Aggregation:** A product's average rating and review count update automatically whenever a review is added or removed.

6. Admin Panel

- **Product Management:** Admins can add, edit, and delete products, including full control over the variant list.
- **Order Management:** Admins can view every order across all customers, filter by status, and update fulfillment status.
- **Site Settings:** Admins manage the homepage banner image and the master list of shopping categories used throughout the storefront.

Roles and Responsibilities

User

- **Registration:** Users create an account on ShopEZ by providing their name, email, and password.
- **Profile Management:** Users can view their account details and their full order history from their profile page.
- **Product Exploration:** Users can browse the catalog, filter by category, search by name, and view detailed product pages with variant-level stock.
- **Cart & Checkout:** Users can add products (with a chosen size/color) to their cart, adjust quantities, and complete checkout with a shipping address and payment method.
- **Order Tracking:** Users can view the status of any order they've placed, from pending through delivery.
- **Providing Feedback:** Users can leave a star rating and written review on products.
- **Logout:** Users can securely log out, clearing their session.

Admin

- **Product Management:** Admins manage the entire product catalog — adding new products with full variant detail, editing existing listings, and removing discontinued items.
 - **Order Management:** Admins view every order placed on the platform, filter by status, and progress each order through its fulfillment lifecycle.
 - **Site Configuration:** Admins manage the homepage banner image and the categories available across the storefront, without needing a code change.
 - **Review Moderation:** Admins can remove any review, in addition to users being able to remove their own.
 - **Inventory Oversight:** Because stock is tracked per variant, admins can see exactly which size/color combinations are running low directly from the product table.
-

User Flow

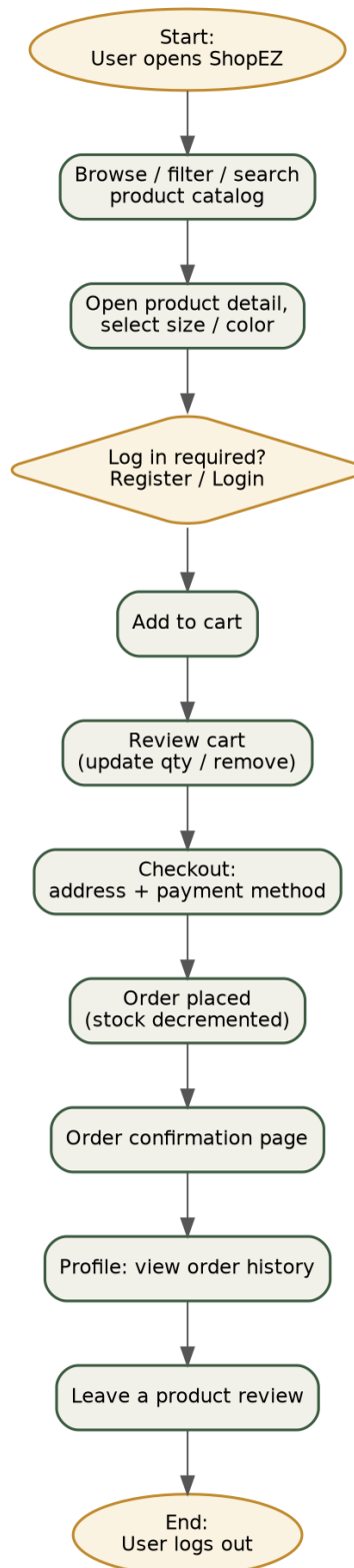


Figure 3: ShopEZ end-to-end user flow

Start: The user opens the ShopEZ application.

Browse: The user browses the product catalog, optionally filtering by category or searching by name.

Product Detail: The user opens a product and selects a size and color combination, seeing live stock for that exact variant.

Authentication (if needed): If the user isn't logged in, adding to cart redirects them to register or log in first.

Add to Cart: The chosen variant and quantity are added to the user's persistent cart.

Review Cart: The user can adjust quantities or remove items before proceeding.

Checkout: The user enters a shipping address and selects a payment method.

Order Placed: Stock is validated and decremented, and an order is created from the cart's contents.

Confirmation: The user sees an order confirmation with the order ID, items, and totals.

Order History: The user can revisit their profile at any time to check an order's status.

Leave a Review: The user can rate and review any product.

End: The user logs out or exits the application.

MVC Pattern

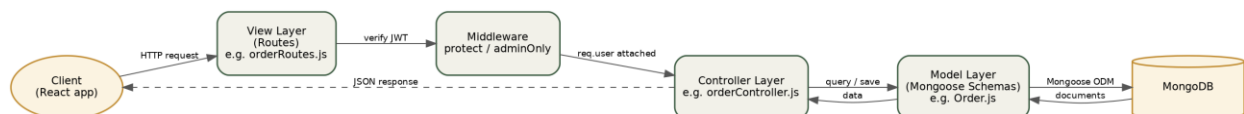


Figure 4: Request flow through ShopEZ's MVC layers

The ShopEZ backend follows the Model-View-Controller (MVC) architectural pattern, separating the application into three interconnected layers for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The Model layer handles all data-related logic, implemented with Mongoose schemas in the server/models folder — User.js, Product.js, Cart.js, Order.js, Review.js, and SiteConfig.js. Each schema defines its fields, validation rules, and any instance methods (for example, User.js includes a comparePassword method and a pre-save hook that hashes passwords automatically).

Controller Layer

The Controller layer, in server/controllers, acts as the intermediary between routes and models. Each controller function receives the incoming request, validates input, performs the necessary business logic (such as checking stock before placing an order), calls the appropriate model methods, and returns a response.

View Layer (Routing Layer)

In this REST API context, the “View” is implemented as the routing layer in server/routes. Each route file defines the endpoints for one resource (for example, orderRoutes.js defines POST /api/orders, GET /api/orders/my, and the admin-only GET /api/orders), and wires each endpoint to its controller function — optionally passing it through the protect and adminOnly middleware first.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features (like the review system) were added by creating a new model, controller, and route file, without touching existing ones.
- **Reusability:** Middleware like protect and adminOnly is reused across every protected route in the app.
- **Testability:** Each controller can be tested independently of the routing layer.
- **Collaboration-Friendly:** Multiple developers could work on different resources (products, orders, reviews) simultaneously without conflicts.

Project Setup and Configuration

Creating the Project Folder

- Create a new folder named SHOPEZ.
- Inside it, create two folders: client and server.
- Open the SHOPEZ folder in Visual Studio Code.

Client Setup (React + Vite)

```
cd SHOPEZ
npm create vite@latest client -- --template react
cd client
npm install
npm install react-router-dom axios
npm install -D tailwindcss @tailwindcss/vite
```

This scaffolds a React app with Vite, then adds React Router for page navigation, Axios for API calls, and Tailwind CSS for styling.

Server Setup (Express + MongoDB)

```
cd SHOPEZ
mkdir server && cd server
npm init -y
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
npm install -D nodemon
```

After installing, package.json needs two manual additions: "type": "module" (to enable ES module import/export syntax), and dev/start scripts:

```
"type": "module",
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Create Database Connection

A dedicated config/db.js file establishes the MongoDB connection before the server starts handling requests:

```
import mongoose from "mongoose";
```

```
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;
```

Create Schemas & Models

Using the entities from the ER Diagram, each collection is defined as a Mongoose schema. Two representative examples:

User.js — defines the schema for user accounts, including automatic password hashing:

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["user", "admin"], default: "user" },
  address: { street: String, city: String, ... },
}, { timestamps: true });

userSchema.pre("save", async function () {
  if (!this.isModified("password")) return;
  this.password = await bcrypt.hash(this.password, 10);
});
```

Product.js — defines the schema for catalog items, with an embedded variant sub-schema:

```
const variantSchema = new mongoose.Schema({
  size: { type: String, default: "N/A" },
  color: { type: String, default: "N/A" },
  stock: { type: Number, required: true, default: 0 },
  priceOverride: { type: Number },
});

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  gender: { type: String, enum: ["Men", "Women", "Unisex", "Kids"] },
  basePrice: { type: Number, required: true },
  discountPercent: { type: Number, default: 0 },
  images: [String],
  variants: [variantSchema],
  ratingAverage: { type: Number, default: 0 },
  numReviews: { type: Number, default: 0 },
}, { timestamps: true });
```

Front-End Development

Frontend Structure

```
client/src/
|-- api/
|   |-- axios.js          (shared axios instance + JWT interceptor)
```

```

|-- context/
|   |-- AuthContext.jsx    (global auth state: login, register, logout)
|-- components/
|   |-- Navbar.jsx
|   |-- ProductCard.jsx
|   |-- ProductReviews.jsx
|   |-- ProtectedRoute.jsx
|-- pages/
|   |-- Home.jsx, About.jsx
|   |-- Login.jsx, Register.jsx
|   |-- Products.jsx, ProductDetail.jsx
|   |-- Cart.jsx, Checkout.jsx, OrderDetail.jsx
|   |-- Profile.jsx
|   |-- admin/
|       |-- AdminDashboard.jsx
|       |-- AdminProducts.jsx, NewProduct.jsx, EditProduct.jsx, ProductForm.jsx
|       |-- AdminOrders.jsx
|       |-- AdminSettings.jsx
|-- App.jsx                (route definitions)
|-- main.jsx               (entry point; wraps App in Router + AuthProvider)
|-- index.css              (Tailwind import + design tokens)

```

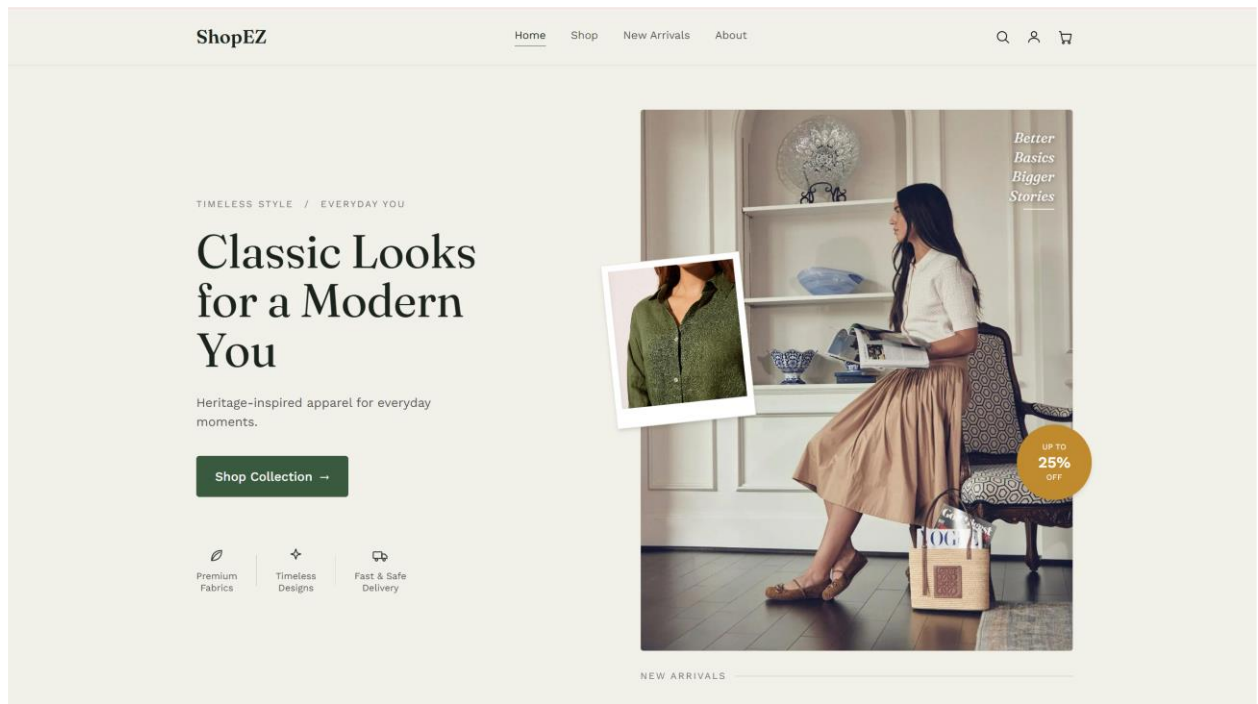
Folder & File Explanations

- **api/axios.js:** A single configured Axios instance that automatically attaches the logged-in user's JWT to every request via an interceptor.
 - **context/AuthContext.jsx:** Provides global authentication state — the current user, and login/register/logout functions — accessible from any component via the `useAuth()` hook.
 - **components/:** Reusable UI pieces used across multiple pages — the Navbar (with live cart count), ProductCard (catalog grid item), ProductReviews (star ratings + review form), and ProtectedRoute (a route guard for logged-in-only and admin-only pages).
 - **pages/:** One component per route, covering the full customer journey (browse → detail → cart → checkout → order → profile) plus a dedicated admin/ subfolder for the entire back-office.
 - **App.jsx:** Defines every route using React Router, wrapping protected and admin-only routes in `<ProtectedRoute>`.
 - **index.css:** Imports Tailwind CSS and defines the app's design tokens (colors, fonts) as CSS custom properties, so the visual theme can be updated from a single file.
-

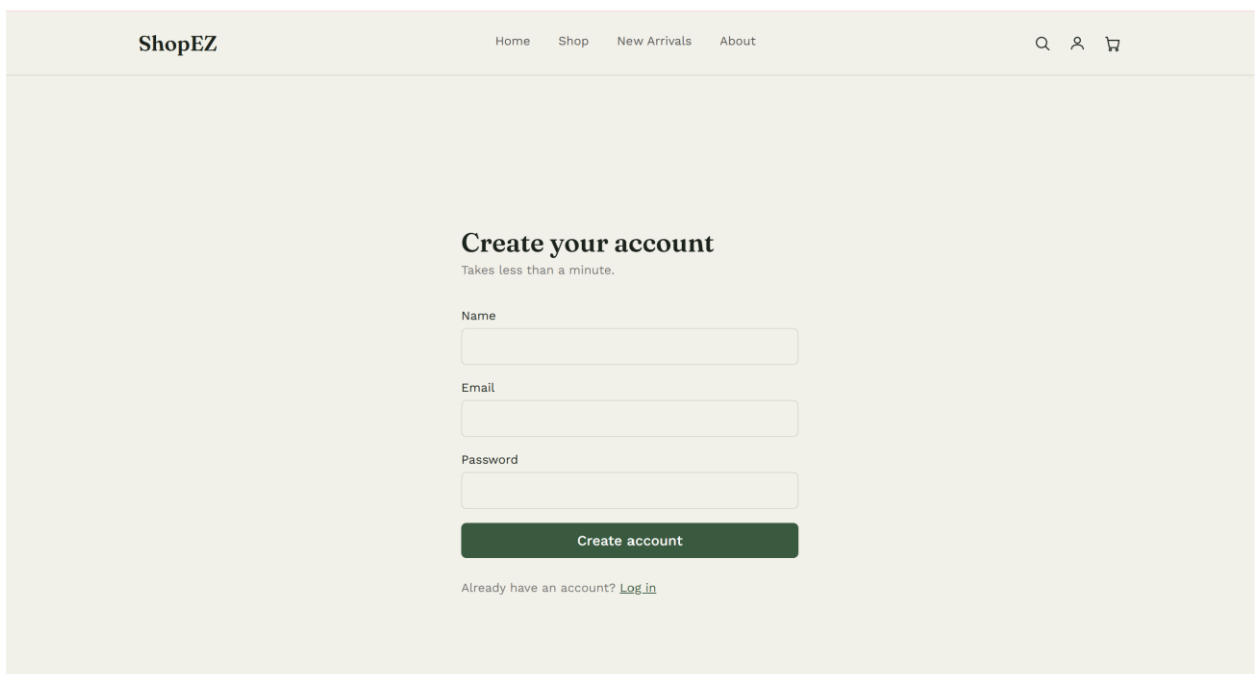
Output Screenshots

The following pages should be captured from the running application and inserted below for the final submission.

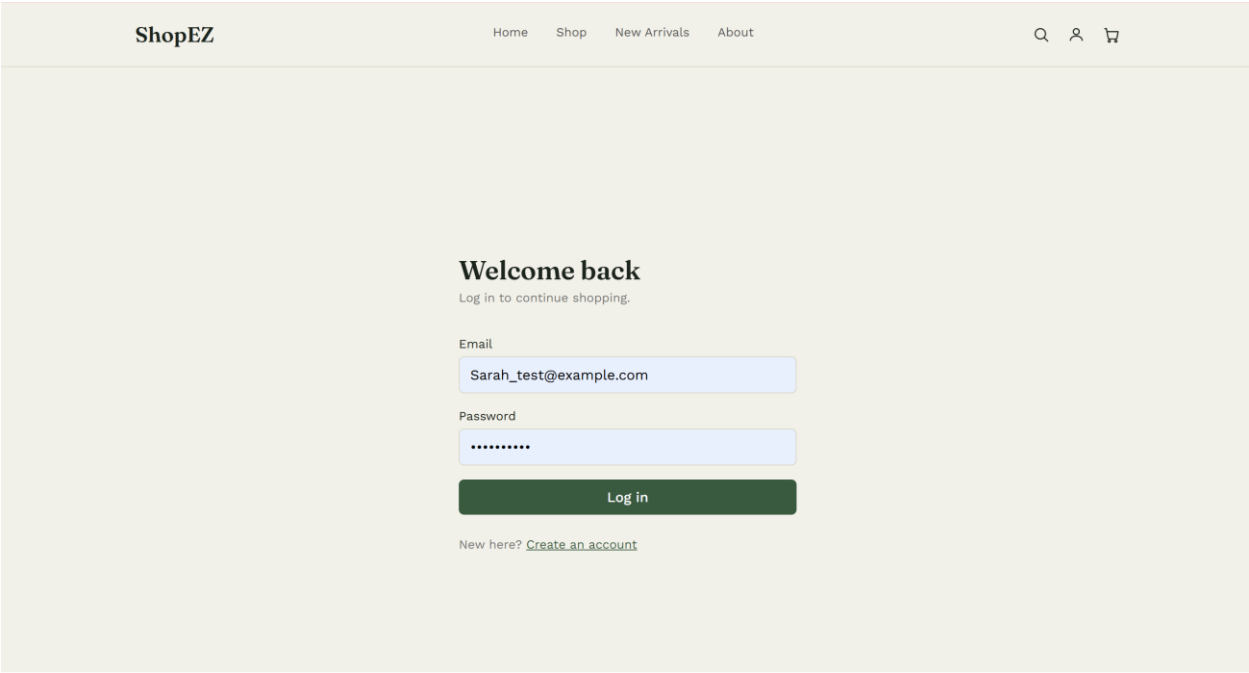
Landing Page



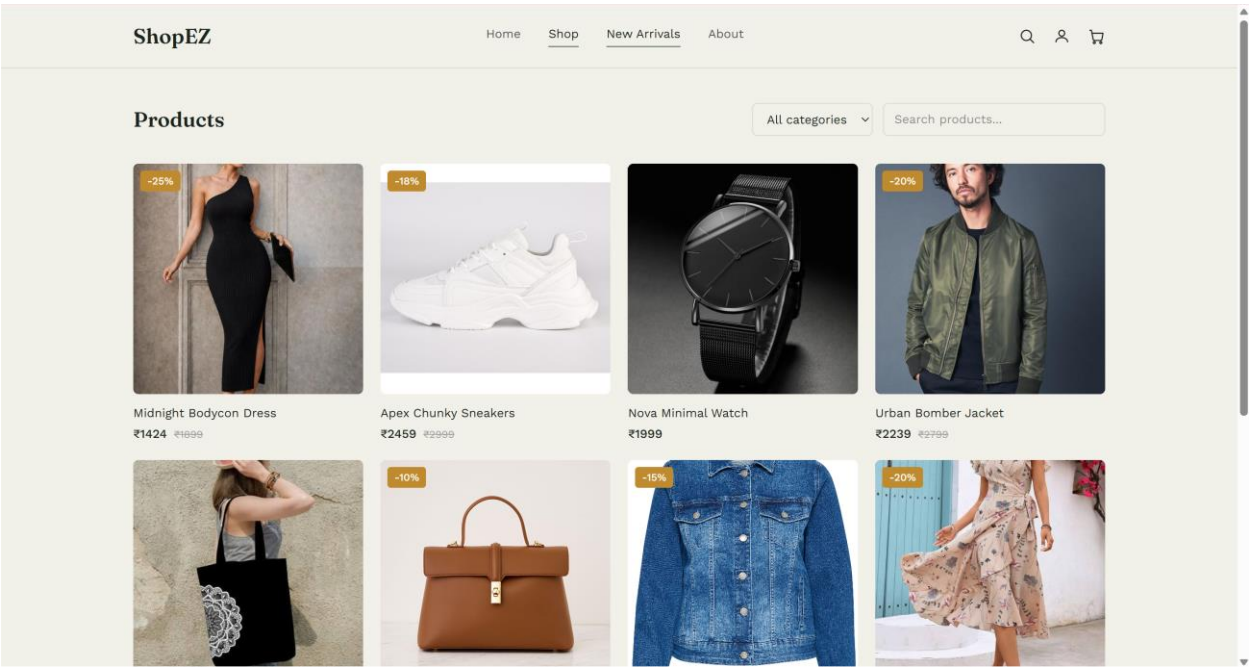
Register Page



Login Page



Products Page (with category filter)



Product Detail Page (variant selection + reviews)

ShopEZ

[Home](#)[Shop](#)[New Arrivals](#)[About](#)

Midnight Bodycon Dress

★★★★★ 5.0 (1)

₹1424 ~~₹1899~~ -25%

Stylish fitted bodycon dress with a sleek silhouette for parties and evening events.

Size

S

M

L

XXL

Color

Black

Black

Quantity

1

17 in stock

Add to cart

Reviews

★★★★★ 5.0 (1 review)

Black

Black

Quantity

1

17 in stock

Add to cart

Reviews

★★★★★ 5.0 (1 review)

Leave a review

★ ★ ★ ★ ★

What did you think of this product?

Log in to review

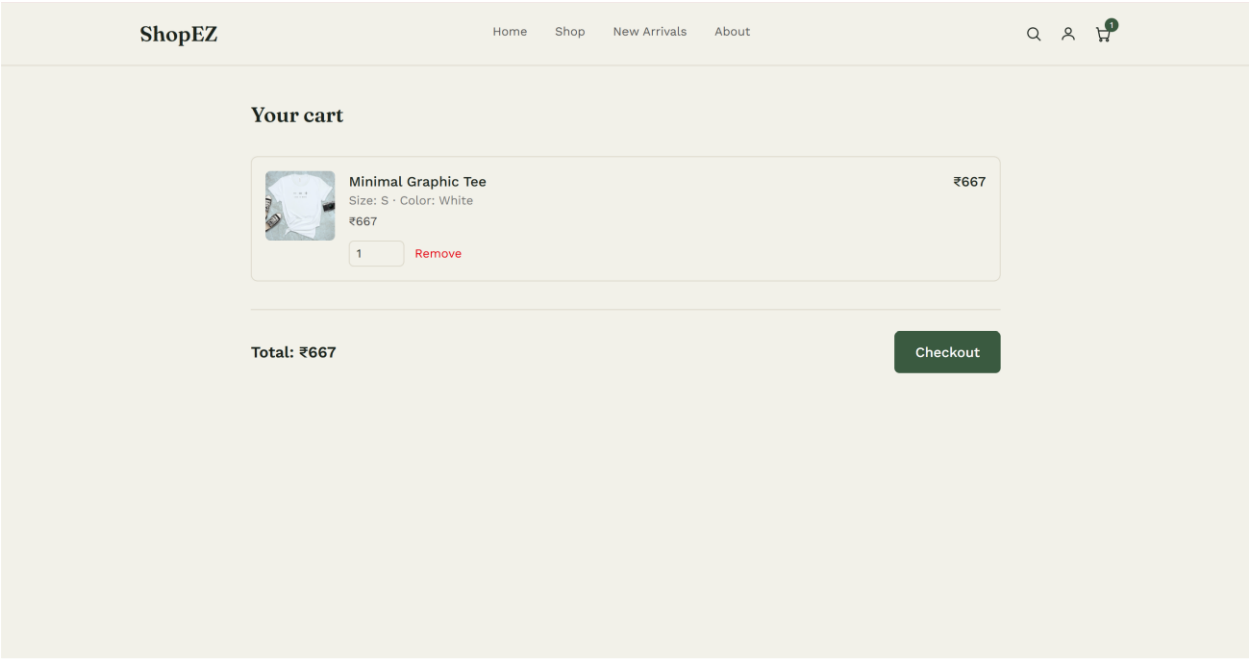
Arya Sakore

★★★★★

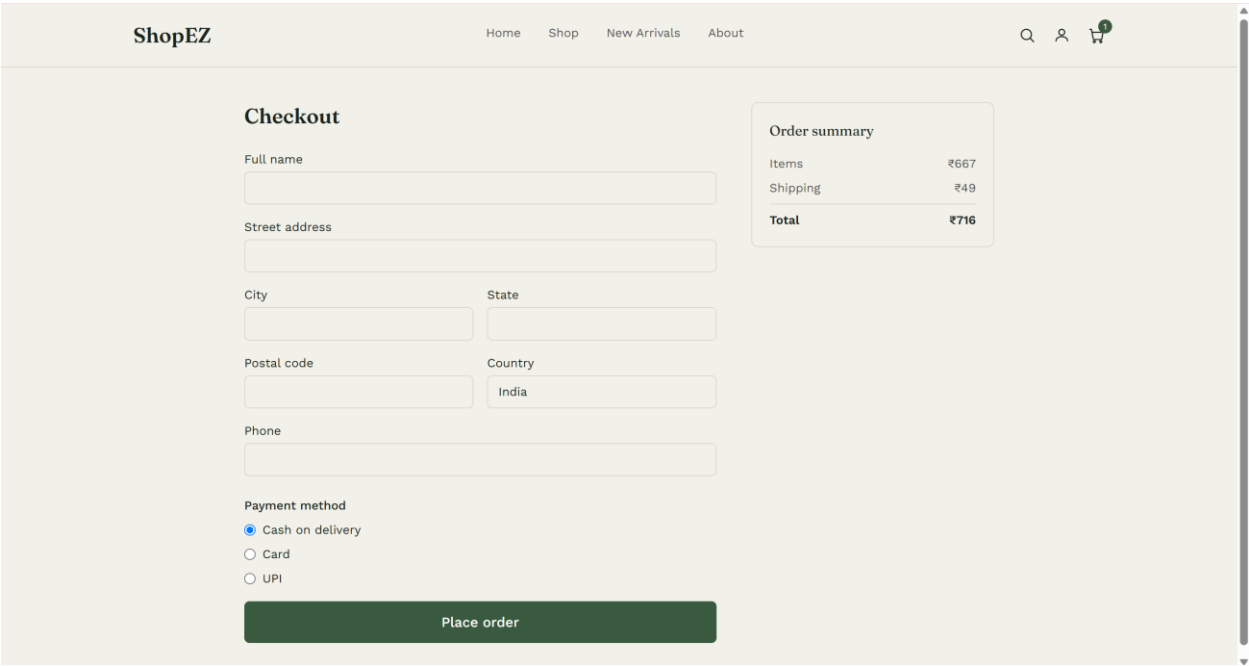
It's smooth and fits like glove!!

9/17/2026

Cart Page



Checkout Page



Order Confirmation Page

ShopEZ

[Home](#)[Shop](#)[New Arrivals](#)[About](#)

Order confirmed

Order ID: 6aad534fae7be91ef73e5702 · Placed 9/18/2026



Minimal Graphic Tee

Size: S · Color: White

₹600 × 1

₹600

Shipping to

Sarah Lopez
Chinchwad
pune, Maharashtra 411033
India
9087654321

Payment

Method: COD
Status: Not paid

Items	₹600
Shipping	₹49
Total	₹649

[Continue shopping](#)

[View all orders](#)

Profile Page (order history)

ShopEZ

[Home](#)[Shop](#)[New Arrivals](#)[About](#)

Your profile

Name: Sarah

Email: sarah_test@example.com

Order history

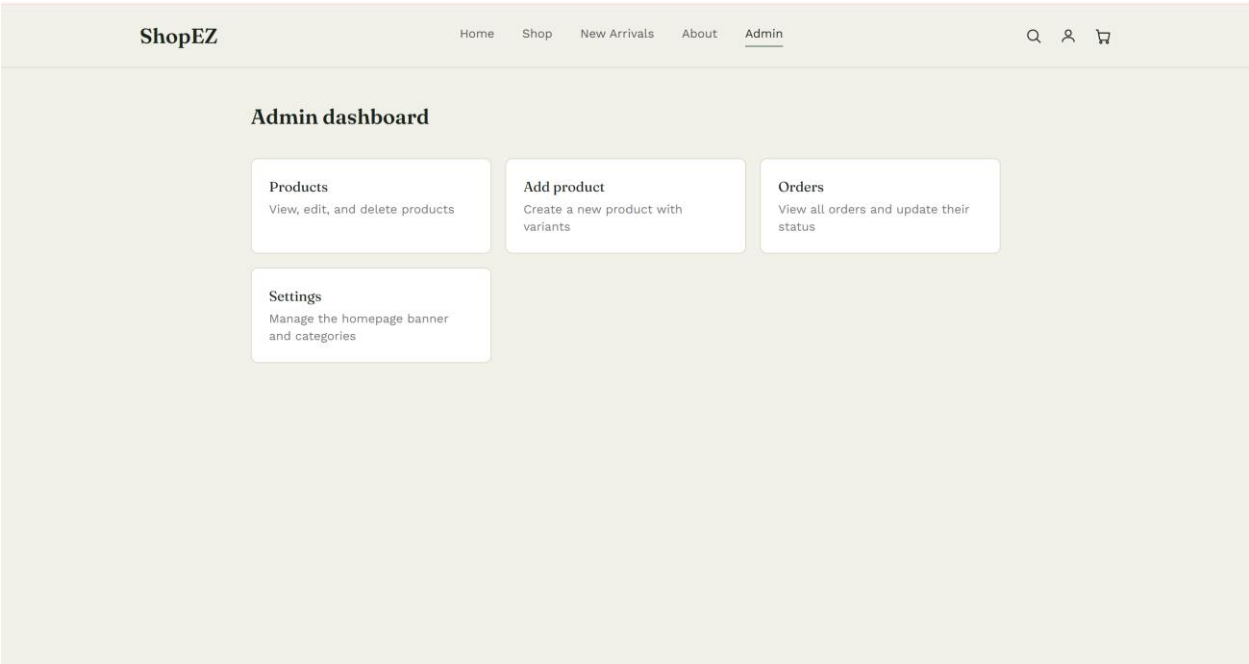
1 item · ₹649

9/18/2026 · COD

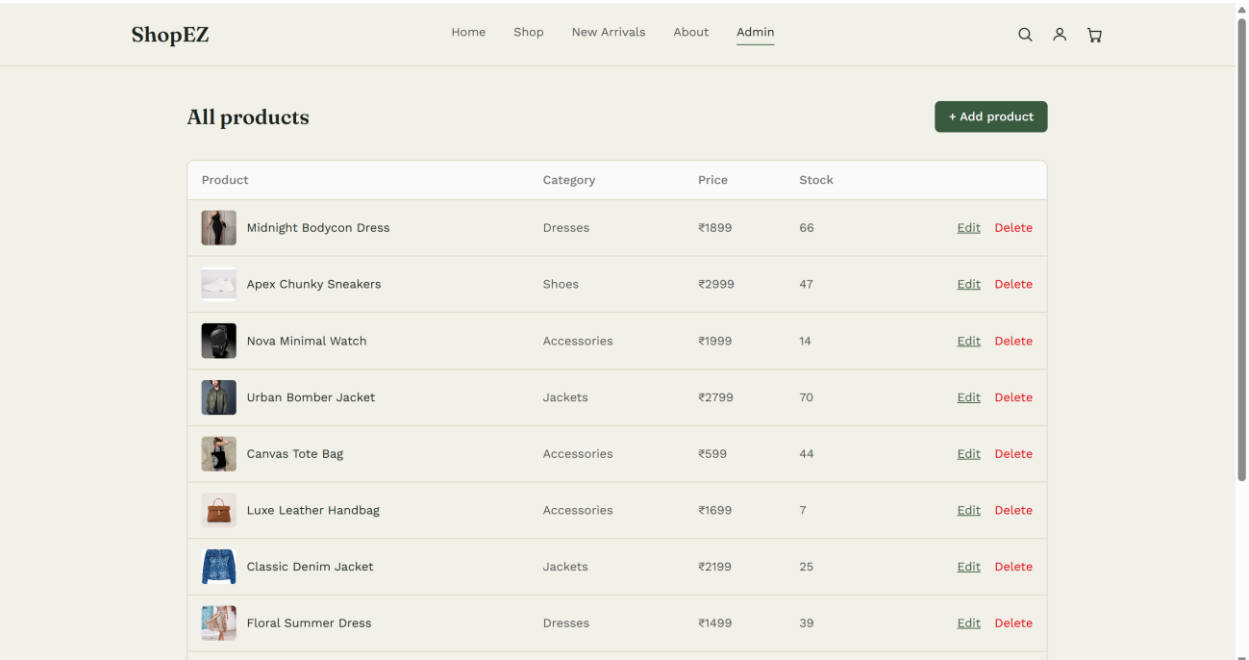
#f73e5702

pending

Admin Dashboard



Admin: All Products



Admin: Add/Edit Product

ShopEZ

Home

Shop

New Arrivals

About

Admin

Q

Person icon

Shopping cart icon

Add a new product

Product name

Midnight Chelsea Boots

Description

Elegant matte black Chelsea boots designed for everyday wear. Featuring a clean silhouette, cushioned insole, and side elastic panels for effortless comfort and timeless styling.

Category

Shoes

Gender

Women

Base price (₹)

1999

Discount % (optional)

15

Image URLs (one per line)

6LPi3HAoDu4-nleec0Md4Fu3DIAxGgNYnsVRIK69Luh2nk5y746zYASHnqqLDcfansBskiJNNbfVXpBT-PDACXN65SrBKmiyvbV4?purpose=inline

Variants

+ Add variant

Shoes

Women

Base price (₹)

1999

Discount % (optional)

15

Image URLs (one per line)

6LPi3HAoDu4-nleec0Md4Fu3DIAxGgNYnsVRIK69Luh2nk5y746zYASHnqqLDcfansBskiJNNbfVXpBT-PDACXN65SrBKmiyvbV4?purpose=inline

Variants

+ Add variant

UK36	Black	11	Price override	Remove
UK37	Black	10	Price override	Remove
UK38	Black	12	Price override	Remove
UK39	Black	9	Price override	Remove
UK40	Black	7	Price override	Remove
UK41	Black	8	Price override	Remove

Leave size or color blank for products without that option. Price override is optional — leave blank to use the base price.

Create product

ShopEZ

[Home](#)[Shop](#)[New Arrivals](#)[About](#)[Admin](#)

Q

Edit product

Product name

Midnight Chelsea Boots

Description

Elegant matte black Chelsea boots designed for everyday wear. Featuring a clean silhouette, cushioned insole, and side elastic panels for effortless comfort and timeless styling.

Category

Shoes

Gender

Women

Base price (₹)

1999

Discount % (optional)

15

Image URLs (one per line)

https://images.openai.com/static-rsc-4/1WGjsN9yuBSvLKrwfwI0T_txSsmpkrfBycDbfzJywAfUTrdoVJGGO64TuRiFn7wgQCgPH

Variants

+ Add variant

Admin: All Orders

ShopEZ

[Home](#)[Shop](#)[New Arrivals](#)[About](#)[Admin](#)

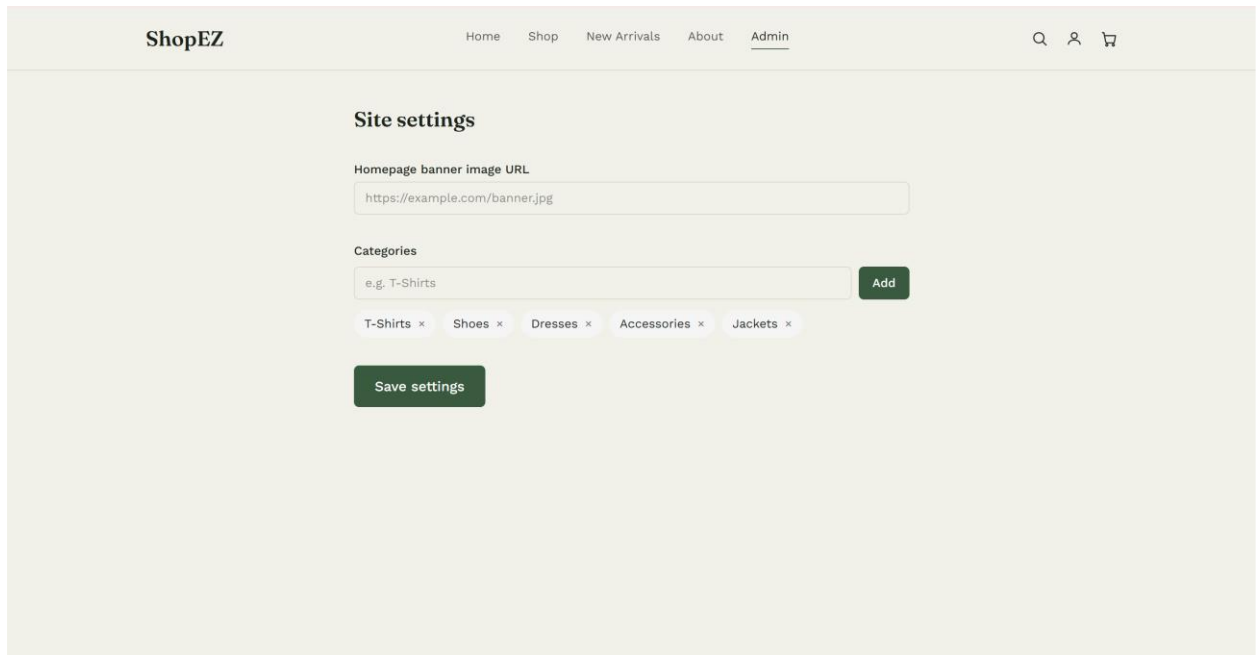
Q

All orders

All statuses

#f73e5702	pending	pending
Sarah · sarah_test@example.com		
1 item · ₹649 · 9/18/2026		
#6d1801a9	processing	processing
Arya Sakore · arya.sakore24@gmail.com		
1 item · ₹1198 · 9/17/2026		

Admin: Settings (banner + categories)



Demo Video & Code Links

Demo Video Link: [Arya Sakore-ShopEZ E-commerce Project.mp4](#)

Code Repository Link: [Arya-24/ShopEZ](#)